

졸업 프로젝트 8월 1차 격주보고서		
SNN을 활용한 고효율 추천 시스템 개발 및 최적화 연구		
2020058259 이승현 2020061676 이강혁		지도교수 김상욱 교수님
지난 2주간 진행사항	1. NCF 모델 구조 개선 및 최적화 2. SNN 기본모델 구현 3. alibaba 데이터 전처리	△ ○ △
진행 내용	1. ANN to SNN선정한 이유 ANN으로 학습한 뒤 SNN으로 변환하여 추론하는 방식을 채택한 이유는 높은 성능과 에너지 효율 두 가지 목표를 모두 달성하기 위해서입니다. 먼저 ANN은 높은 정확도를 갖지만, 그만큼 많은 전력을 필요로 합니다. 기본적으로 실수 연산과 곱연산이 사용되고, 데이터와 가중치의 형태가 행렬이므로 행렬 연산 또한 사용됩니다. 이와 달리 SNN은 이산, 즉 0,1로 이루어진 데이터를 사용하며 합연산만을 사용합니다. 각 뉴런마다 발화의 유무만이 다음 뉴런에 영향을 미치므로 layer 간의 행렬 연산 또한 없습니다. 또한, 학습과 추론의 과정에서 모델의 뉴런이 모두 활성화되는 ANN과 달리, SNN은 spike가 발화할 때만 뉴런이 활성화되는 이벤트 기반 방식으로 작동합니다. 이러한 이유로 SNN을 사용했을 때 ANN에 비해 분명한 에너지 효율성을 가질 수 있습니다. 하지만, 이러한 전력량 문제를 해결하기 위해 SNN을 통해 학습과 추론 모두를 하려고 한다면 문제가 발생합니다. 그 이유는 이산 데이터의 불연속성 때문입니다. 이는 역전파의 과정에서 미분이 불가능하다는 문제가 됩니다. 이를 보완하기 위한 역전파 방식들이 연구되어 정확도 면에서 큰 차이가 없는 방식들이 탄생했지만, SNN의 장점이었던 에너지 효율을 희생한다는 문제점이 있습니다. 문제는, 이 희생이 예상보다 더 심각할 수 있다는 것입니다. SNN은 시간에 따른 spike의 누적과 leaky를 계산합니다. 즉, 한 번의 학습마다 시간의 영향을 계산하기 위해 timestep	

이라는 새로운 과정이 추가됩니다. 이러한 timestep이 클수록 더많은 연산 과정이 발생하게 됩니다. 시간에 따른 메모리 접근 (leaky를 계산하기 위한 과거 데이터의 저장, 발화 임계치와 비교를 위한 spike 합계 저장) 또한 timestep에 비례하여 많아집니다. 즉, 정확도를 높이기 위해 timestep를 늘리게 되면 연산량과, 특히 메모리 접근이 과도하게 증가하면서 오히려 ANN에 비해 전력 소비가 커지는 문제가 발생합니다.

이와 같은 이유로 저희는 'ANN to SNN 변환' 방식을 사용하기로 결정했습니다. 'ANN to SNN 변환'은 먼저 ANN을 통해 학습을 진행하고 학습된 모델을 저장합니다. 저장된 모델을 SNN의 형식으로 변환하여 neuromorphic chip에서 추론을 실행합니다. 이 방식은 SNN의 학습 문제를 해결하면서도 기존 ANN에 비해 에너지 효율성을 가져갈 수 있게 됩니다. 다만, 이 방식에서는 추론만 SNN으로 진행하기 때문에 에너지 효율이 기존의 ANN에 비해 많이 좋아질지에 대한 의문이 있었습니다. 이에 대한 정보를 조사해보니, 대형 플랫폼을 수많은 유저들이 사용할 때 많은 추론이 발생하므로 추론의 에너지 절감이 의미 있을 것이라 생각되었습니다. 또한, 클라우드 시스템을 통해 모델을 사용하던 스마트폰, IoT 같은 환경에서 사용한다면, 학습된 모델은 클라우드를 통해, 추론은 저전력으로 내부 환경에서 진행할 수 있다는 장점이 있습니다.

2. sPyNNaker선정 이유

Neuromorphic chip을 사용하여 SNN을 실행하기 위해 neuromorphic chip을 컨트롤할 수 있도록 만들어진 sPyNNaker를 사용하기로 하였습니다. SNN을 지원하는 다른 라이브러리들은 pytorch와 연계되어 편리하다는 장점이 있지만, neuromorphic chip을 인식하여 사용하는 것이 아닌 neuromorphic chip의 환경을 모방한 자체적인 에뮬레이터를 사용합니다. Neuromorphic chip을 사용할 수 있는 라이브러리는 sPyNNaker가 거의 유일하였고, 저희가 원격으로 neuromorphic chip을 사용할 수 있게 해주는 Ebrains에서도 sPyNNaker 주로 사용할뿐더러, sPyNNaker가 자원하는 전력 측정 함수가 있기에 sPyNNaker를 사용하기로 결정하였습니다.

3. 진행 방법

EBRAINS Lab의 Jupyter Notebook에서 PyNN 코드를 작성하고 실행하면, 자동으로 SpiNNaker 하드웨어와 연결되어 시뮬레이션이 진행됩니다.

(1) 전력량 확인방법

SpiNNaker 소프트웨어는 에너지 소비량을 추정할 수 있습니다. 가장 쉬운 방법은 시뮬레이션을 실행하는 폴더에 spynnaker.cfg 파일을 추가하는 것입니다.

해당 파일에 power_reports와 write_energy_report를 넣게 되면, power_reports는 실시간 전력을 기록하고, write_energy_report는 그 과정을 바탕으로 한 총에너지를 계산하게 됩니다.

에너지 사용량 추정 리포트는 시뮬레이션 실행 후 확인할 수 있습니다.

이 수치는 과거 전력 측정 장치를 사용하여 SpiNNaker 보드에서 측정된 실제 데이터 기반으로 산출됩니다.

4. 진행 내용

(1) 기본적인 SNN모델 구현

- 뉴런 모델

단순화된 LIF (Leaky Integrate-and-Fire) 뉴런을 사용

막전위 업데이트, 임계값(threshold) 도달 시 발화(spike) 및 리셋(reset) 메커니즘 구현

- 네트워크 구조

입력층 - 은닉층(스파이킹 뉴런) - 출력층으로 단순 구성

은닉층에서는 스파이크 발생, 출력층에서는 spike count 또는 발화율을 출력 값으로 사용

- 학습 방식

surrogate gradient 기법을 활용하여 역전파 학습

(2) ANN to SNN 변환을 위한 선행 작업 진행중 (변환 친화적 설계)

- 모델 구조 제약

활성화 함수는 ReLU 함수 사용 (스파이킹 발화와 대응하기 위함)

BatchNorm, Dropout 등은 변환 과정에서 제약이 크므로 최소화 필요

- 출력 및 가중치 정규화

각 레이어 출력의 크기를 일정 범위로 정규화하여 스파이크 발화가 과도하거나 부족하지 않도록 조정 필요

- 데이터 인코딩 고려

ANN은 연속값을 입력으로 하지만, SNN은 스파이크로 표현해야 한다.

Poisson rate coding, direct rate coding 등 변환 전략을 염두에 두고 ANN을 설계 필요

진행 중 이슈	뇌 신경의 학습 방식은 (1) 학습에 따라 신경 세포가 방향에 맞게 자라거나 없어지는 방식으로 학습합니다. (2) 또한, 특정 신경 신호를 강화하거나 저하하는 방식으로 작동합니다. 기존의 ANN, SNN과 다르게 layer 수와 뉴런 수가 고정된 것이 아닌, “학습에 따라 유동적으로 변화하는 방식의 학습을 사용한다면 어떨까?”라는 생각을 했습니다. 기존의 딥러닝 기반 SNN이 주로 사용되는 분야는 그래픽이었습니다. 그래픽 데이터는 보통 dense하지만, 저희가 적용하려는 추천시스템 분야의 데이터는 일반적으로 매우 sparse합니다. 그래서 추천시스템은 일반적으로 drop-out을 통해 fully-connective하지 않도록 모델을 만들고, sparse한 데이터를 embedding하여 사용합니다. 이런 점에서 기존의 layer 기반 딥러닝 구조보다는 map 형태의 구조가 유리하지 않을까 생각했습니다. 또한, map 구조를 사용한다면 SNN으로도 충분히 많은 데이터를 표현할 수 있을 것이라 생각합니다. 이는 기존 SNN이 학습 방식의 한계점으로 인해 주로 predict에서만 사용되던 부분을 개선할 수 있을 것이라고 생각했었습니다. => 이와 관련된 논문을 찾아본 결과 GCN과 SNN을 결합한 논문들이 존재하였고, 추천시스템을 대상으로 논문을 작성한 것은 아니지만, 한 논문에서 추천시스템에서도 좋은 결과를 보였다는 내용이 있어 보류하기로 결정했습니다.
다음 2주간 계획	ANN to SNN 변환을 위한 ANN 선행 작업